

You are a senior full-stack Python developer, Machine Learning engineer, NLP engineer, and UI/UX designer.

Build a complete, production-quality academic AIML mini-project called:

"DocuGuard – Intelligent Fake Document Detection & Verification System"

The project is being developed as a college AIML mini-project by a team of 3–4 students.

The application must satisfy the following academic requirement:

"Design and develop an application or decision support system for the selected problem statement from domains such as Security & Surveillance. Visualize the results and perform a comparative performance analysis of at least three different machine learning models or deep learning models using appropriate evaluation methods."

The selected domain is:

SECURITY & SURVEILLANCE

Problem:

Detect whether an uploaded document appears genuine or potentially fraudulent by analyzing its textual, structural, formatting, metadata, and document-level characteristics.

IMPORTANT:

Do NOT build a simplistic keyword classifier.

The system must analyze the entire uploaded document and provide explainable evidence for its decision.

The system must NOT claim absolute authenticity.

The final result should be expressed as:

- Likely Genuine
- Suspicious
- Likely Fake

with a confidence/risk score and detailed evidence.

=====

1. CORE OBJECTIVE

=====

Create a web-based AI/ML application where a user can upload a document and the system:

1. Accepts documents in multiple formats.
2. Extracts the complete document content.
3. Performs OCR when necessary.
4. Analyzes textual content.
5. Detects suspicious linguistic patterns.

6. Detects suspicious numerical/date inconsistencies.
7. Detects unusual document structure.
8. Detects suspicious formatting patterns.
9. Extracts document metadata when available.
10. Identifies unusual markers/signatures/fields.
11. Detects duplicated or inconsistent information.
12. Generates document-level features.
13. Runs multiple ML models.
14. Compares model predictions.
15. Produces a final risk score.
16. Explains why the document was classified as suspicious or genuine.
17. Displays visual analytics.
18. Stores scan history locally/database-backed.
19. Allows users to download a detailed analysis report.

The application should look like a serious cybersecurity/document-forensics product rather than a generic AI demo.

=====

2. IMPORTANT AI/ML REQUIREMENT

=====

The project MUST implement and compare at least THREE machine learning/deep learning models.

Recommended models:

MODEL 1:
Logistic Regression

MODEL 2:
Random Forest Classifier

MODEL 3:
XGBoost / Gradient Boosting

OPTIONAL MODEL 4:
SVM

OPTIONAL DEEP LEARNING MODEL:
BERT/DistilBERT text classifier

If computational requirements are high, use TF-IDF + traditional ML models as the main academic comparison and optionally implement DistilBERT as an advanced model.

The project must display comparative performance.

Metrics:

- Accuracy
- Precision
- Recall
- F1 Score
- ROC-AUC
- Confusion Matrix

Generate:

- Model comparison bar chart
- Confusion matrix
- ROC curve
- Precision/Recall comparison
- Training time comparison where possible

Do not fabricate performance values.

If a dataset is not available, create the project architecture so a dataset can be added through:

data/documents.csv

with columns such as:

document_id
text
label

where:

label = 0 → Genuine
label = 1 → Fake

Also support a richer dataset containing extracted document features.

=====

3. DOCUMENT INPUT FORMATS

=====

The application should support:

PDF
DOCX
DOC
TXT
PNG
JPG

JPEG
TIFF

Potential libraries:

PyMuPDF / fitz
python-docx
Pillow
pytesseract
OpenCV
pdf2image

For PDF:

- extract text directly when available
- identify scanned/image-only pages
- perform OCR on image pages

For images:

- preprocess using OpenCV
- grayscale
- denoise
- threshold
- deskew when possible
- OCR using Tesseract

For DOCX:

- extract paragraphs
- tables
- headers
- footers
- document properties

For TXT:

- directly parse text

The system should display:

File name
File type
File size
Number of pages
Number of words
Number of characters
Number of detected tables

OCR used or not
Metadata availability

=====

4. DOCUMENT ANALYSIS PIPELINE

=====

Implement the following pipeline:

```
graph TD
    A[UPLOAD] --> B[FILE VALIDATION]
    B --> C[DOCUMENT PARSING]
    C --> D[OCR IF REQUIRED]
    D --> E[TEXT NORMALIZATION]
    E --> F[STRUCTURAL ANALYSIS]
    F --> G[FEATURE EXTRACTION]
    G --> H[MARKER DETECTION]
    H --> I[ML MODEL PREDICTION]
    I --> J[RISK SCORING]
    J --> K[EXPLAINABILITY ENGINE]
    K --> L[FINAL DECISION]
    L --> M[VISUAL REPORT]
```

Each step should be represented internally as a modular Python component.

Recommended structure:

```
app/
  main.py
  config.py

api/
  upload.py
  analysis.py
  models.py
  reports.py
```

core/
document_processor.py
ocr_engine.py
text_analyzer.py
marker_detector.py
feature_engineering.py
risk_engine.py
explainability.py

models/
logistic_model.py
random_forest.py
xgboost_model.py
bert_model.py

utils/
file_utils.py
metadata.py
validators.py

database/
models.py
database.py

frontend/

data/
raw/
processed/
training/

models/
saved/

reports/

tests/

=====

5. UNIQUE MARKER DETECTION

=====

This is one of the most important features.

The system should identify "document markers".

A marker is an observable characteristic that can help determine whether a document is suspicious.

Create a Marker Detection Engine.

Categories:

A. TEXTUAL MARKERS

Examples:

- inconsistent terminology
- unusual grammar
- suspicious phrases
- repeated sentences
- abnormal capitalization
- excessive punctuation
- inconsistent spelling
- suspicious template-like language
- unusual spacing
- inconsistent names
- inconsistent organization names

Do NOT simply label documents fake because of grammar.

Treat these as supporting evidence only.

B. NUMERICAL MARKERS

Detect:

- inconsistent totals
- subtotal + tax mismatch
- amount mismatch
- percentage inconsistencies
- repeated numbers that differ
- suspicious account/reference number patterns
- inconsistent IDs

Example:

Subtotal: ₹10,000

Tax: ₹1,800

Total: ₹11,500

Flag:

"Numerical inconsistency detected."

C. DATE MARKERS

Detect:

- impossible dates
- issue date after expiry date
- expiry date before issue date
- inconsistent dates across pages
- unusual date formats
- future issue dates where inappropriate
- inconsistent date references

D. ID / REFERENCE MARKERS

Detect patterns such as:

- document number
- invoice number
- registration number
- application ID
- certificate number
- reference number

Check:

- repeated IDs
- inconsistent IDs
- malformed patterns
- unexpected character combinations

Do NOT connect to external government databases unless an official API is explicitly configured.

E. STRUCTURAL MARKERS

Analyze:

- missing expected sections
- unusual page order

- inconsistent headers
- inconsistent footers
- missing page numbers
- abnormal page numbering
- repeated pages
- blank pages
- inconsistent margins
- text alignment anomalies
- unusual font changes
- unusual spacing

F. VISUAL MARKERS

For image/PDF documents:

Use OpenCV/PIL to detect:

- unusual image regions
- suspicious overlays
- inconsistent background regions
- excessive compression artifacts
- abrupt text/image boundaries
- potential copy-paste regions
- inconsistent resolution
- suspiciously edited areas

IMPORTANT:

Do not claim that image artifacts definitively prove forgery.

Use language such as:

"Potential visual alteration detected."

G. METADATA MARKERS

Extract:

PDF metadata:

- Author
- Creator
- Producer
- Creation date
- Modification date
- PDF version

DOCX metadata:

- Author
- Last modified by
- Creation date
- Modification date

Check for:

- suspicious modification timestamps
- missing metadata
- inconsistent creator information
- editing software information

Metadata should be treated as supporting evidence, not definitive proof.

=====

6. DOCUMENT TYPE DETECTION

=====

Automatically identify document category where possible.

Possible categories:

- Invoice
- Certificate
- Identity document
- Educational document
- Employment document
- Bank/financial document
- Receipt
- Application form
- Agreement
- Letter
- Government-style document
- Unknown

Use NLP/classification rules or a trained model.

Document type detection should influence expected markers.

For example:

Invoice:

Expected:

- invoice number
- date

- seller
- buyer
- subtotal
- tax
- total

Certificate:

Expected:

- issuing organization
- recipient name
- date
- certificate title
- certificate/reference number
- signature/seal area

Do not hard-code fake conclusions based on missing fields alone.

=====

7. FEATURE ENGINEERING

=====

Create a feature extraction system.

Possible features:

TEXT FEATURES

- word count
- character count
- sentence count
- average sentence length
- vocabulary richness
- punctuation ratio
- uppercase ratio
- digit ratio
- special character ratio
- repeated phrase count
- spelling anomaly count
- suspicious phrase count

STRUCTURAL FEATURES

- page count
- paragraph count
- table count
- heading count
- header count

- footer count
- blank page count
- font count
- font size variance
- alignment variance
- page number consistency

NUMERICAL FEATURES

- number count
- monetary values
- date count
- total inconsistencies
- ID inconsistencies

OCR FEATURES

- OCR confidence
- low-confidence text percentage
- detected text regions

METADATA FEATURES

- metadata availability
- creator information
- modification information
- author information

VISUAL FEATURES

- image count
- image resolution statistics
- compression indicators
- suspicious region count

Create a final feature vector.

=====

8. MACHINE LEARNING PIPELINE

=====

Use scikit-learn.

Preprocessing:

1. Clean text.
2. Normalize text.
3. TF-IDF vectorization.

4. Combine textual and numerical/structural features where appropriate.

5. Split dataset:

80% training

20% testing

Use stratified split.

For model evaluation, optionally use:

5-fold Stratified Cross Validation.

Models:

1. Logistic Regression

2. Random Forest

3. XGBoost / Gradient Boosting

If XGBoost is unavailable, use HistGradientBoostingClassifier or GradientBoostingClassifier.

Optional:

DistilBERT.

Save trained models using:

joblib

or appropriate Hugging Face/PyTorch serialization.

Do NOT retrain models every time a document is uploaded.

Training should be a separate process.

=====
9. ENSEMBLE / FINAL DECISION ENGINE
=====

The final result should NOT depend solely on one model.

Generate:

model probabilities

Example:

Logistic Regression:

Fake probability = 0.72

Random Forest:
Fake probability = 0.81

XGBoost:
Fake probability = 0.77

Combine model probabilities.

Also incorporate marker evidence.

Example conceptual score:

ML Risk Score
+
Marker Risk Score
+
Structural Risk Score
+
Consistency Risk Score

Then normalize to:

0–100 Risk Score

Classification:

0–29:
Likely Genuine

30–59:
Suspicious

60–100:
Likely Fake

IMPORTANT:

Clearly explain that this is an AI-assisted risk assessment and not legal/documentary authentication.

=====
10. EXPLAINABLE AI
=====

The system **MUST** explain its decision.

Do not only show:

"Fake: 87%"

Instead show:

"Risk Score: 78/100"

Then:

Detected Evidence:

1. Date inconsistency
Issue date occurs after expiry date.
2. Numerical inconsistency
Total does not match calculated subtotal + tax.
3. Formatting anomaly
One section uses a substantially different font size.
4. Textual anomaly
Repeated phrase detected.
5. Metadata anomaly
Modification timestamp differs significantly from creation timestamp.

For ML explainability:

Use:

SHAP

or

feature_importances_

depending on model.

Display top contributing features.

Example:

Top contributing factors:

Date inconsistency	+18
Number inconsistency	+14
Formatting anomaly	+9
Repeated content	+7

OCR uncertainty +4

=====

11. USER INTERFACE

=====

Build a polished modern cybersecurity-style UI.

COLOR SCHEME:

Primary:

BLACK

Secondary:

DARK CHARCOAL

Accent:

GREEN

Use shades of green for:

- success
- highlights
- charts
- buttons
- progress indicators

Do NOT use:

- glassmorphism
- frosted glass
- excessive gradients
- translucent cards
- neon cyberpunk overload

The UI should be:

- clean
- professional
- minimal
- technical
- modern
- readable

Use:

black background
dark cards

green accent
white/light-gray text
green borders/highlights

Avoid making every element green.

=====

12. LANDING PAGE

=====

Create:

Header:

DocuGuard
"AI-Powered Document Verification"

Navigation:

Dashboard
Analyze Document
Model Performance
History
About

Hero:

"Detect suspicious documents before they become a problem."

Subtitle:

"AI-assisted document analysis using machine learning, OCR, structural analysis and explainable risk scoring."

Primary button:

"Analyze Document"

Secondary:

"View Model Performance"

Include a subtle document scanning animation.

Do not make it visually excessive.

=====

13. DASHBOARD

=====

Dashboard should show:

Total Documents Scanned

Likely Genuine

Suspicious

Likely Fake

Average Risk Score

Most Common Detected Marker

Model Accuracy

Recent Scans

Charts:

Document Classification Distribution

Risk Score Distribution

Marker Frequency

Model Performance

=====

14. DOCUMENT UPLOAD PAGE

=====

Create a large upload area.

Support:

Drag & Drop

Browse Files

Multiple file selection optionally.

Show:

Supported formats:

PDF, DOCX, DOC, TXT, PNG, JPG, JPEG, TIFF

After upload:

file preview
file metadata
processing status

Pipeline animation:

Uploading
Extracting
OCR
Analyzing
Detecting markers
Running ML models
Generating report

=====
15. ANALYSIS RESULT PAGE
=====

This should be the most important page.

Top section:

Document Name

Document Type

Risk Score

Decision

Example:

78 / 100

LIKELY FAKE

Use appropriate visual hierarchy.

Then:

Confidence:
87%

MODEL PREDICTIONS

Logistic Regression

Suspicious Probability: 72%

Random Forest

Suspicious Probability: 81%

XGBoost

Suspicious Probability: 77%

Ensemble:

77%

DETECTED MARKERS

Display cards/table:

Marker

Category

Severity

Evidence

Score

Example:

Date mismatch

Date

High

Issue date occurs after expiry date

+18

Font inconsistency

Formatting

Medium

Page 2 uses an inconsistent font

+8

DOCUMENT STATISTICS

Pages

Words

Characters

Tables

Images
OCR Confidence

EXPLAINABILITY

Show top factors contributing to the result.

Use horizontal bars.

16. DOCUMENT VIEWER

If possible, provide a document preview.

For PDF:

Render pages.

Allow:

- page navigation
- zoom
- highlighted suspicious regions if available

If OCR identifies suspicious text, show:

Highlighted Marker

Example:

[red/green/yellow highlight]

"Date inconsistency"

The application should associate detected markers with:

page number
text snippet
location where possible

17. MODEL PERFORMANCE PAGE

Create an academic model comparison dashboard.

Show:

Model | Accuracy | Precision | Recall | F1 | ROC-AUC

Models:

Logistic Regression

Random Forest

XGBoost

Add charts.

Chart 1:

Accuracy comparison

Chart 2:

Precision / Recall / F1

Chart 3:

ROC curves

Chart 4:

Confusion matrices

Chart 5:

Training time

Add explanation:

"Model performance is measured on the held-out test dataset."

Do not invent numbers.

If the model has not yet been trained, display:

"Training required – no evaluation data available."

=====
18. TRAINING PAGE
=====

Create an admin/developer section.

Allow:

Upload dataset

Select label column

Train models

View training progress

Evaluate models

Save trained models

Display:

Dataset size

Training samples

Testing samples

Class distribution

Important:

Prevent accidental retraining from normal document scanning.

=====

19. HISTORY PAGE

=====

Store previous scans.

Table:

Date

File Name

Document Type

Risk Score

Decision

Top Marker

Allow:

View analysis

Delete scan

Download report

Search

Filter:

Genuine
Suspicious
Fake

=====

20. REPORT GENERATION

=====

Generate a downloadable PDF report.

Report should include:

DocuGuard logo/title

Document information

Scan timestamp

Document statistics

Final risk score

Classification

Model predictions

Detected markers

Evidence

Top ML features

Model performance

Disclaimer

Example disclaimer:

"This report represents an AI-assisted risk assessment based on document characteristics, extracted content, structural properties and trained machine learning models. It does not constitute legal authentication or definitive proof that a document is genuine or fraudulent."

Use ReportLab.

=====

21. BACKEND

=====

Recommended backend:

FastAPI

Python

Endpoints:

POST /api/upload

POST /api/analyze

GET /api/results/{id}

GET /api/history

DELETE /api/history/{id}

GET /api/report/{id}

GET /api/models/performance

POST /api/models/train

GET /api/health

Use Pydantic models.

Add proper:

- error handling
- validation
- file size limits
- allowed file extensions
- temporary file cleanup
- logging

=====

22. DATABASE

=====

Use SQLite for the mini-project.

SQLAlchemy ORM.

Tables:

documents

analyses

markers

model_predictions

model_metrics

Optional:

document_pages

Example documents:

id

filename

file_type

size

pages

uploaded_at

document_type

analyses:

id

document_id

risk_score

decision

confidence

created_at

markers:

id

analysis_id

category

name

severity

description

page_number

evidence

score

model_predictions:

id

analysis_id

model_name
genuine_probability
fake_probability

model_metrics:

id
model_name
accuracy
precision
recall
f1
roc_auc

=====
23. FRONTEND
=====

Use:

React
Vite
Tailwind CSS

OR

Next.js if appropriate.

Recommended:

React + Vite + Tailwind

Charts:

Recharts

Icons:

Lucide React

Document viewer:

react-pdf where suitable.

The frontend should communicate with FastAPI.

=====
24. SECURITY

=====

Since this application processes documents, implement basic security.

Requirements:

- File extension validation
- MIME type validation
- File size limit
- Random temporary filenames
- Sanitize filenames
- Never execute uploaded files
- Never trust document metadata
- Delete temporary files after processing
- Handle malformed PDFs safely
- Avoid exposing filesystem paths

Default maximum upload:

25 MB

Make configurable through environment variables.

=====

25. PROJECT STRUCTURE

=====

Create a clean repository:

docuguard/

backend/

app/

main.py

config.py

api/

upload.py

analysis.py

history.py

reports.py

models.py

core/

document_processor.py

ocr_engine.py

feature_engineering.py

marker_detector.py

risk_engine.py
explainability.py

ml/
 preprocessing.py
 train.py
 evaluate.py
 predict.py

models/
 schemas.py
 database_models.py

database/
 database.py

utils/
 file_validation.py
 logging.py

requirements.txt

frontend/
 src/
 components/
 pages/
 services/
 hooks/
 utils/
 App.jsx
 main.jsx

data/
 raw/
 processed/
 training/

ml_models/

reports/

tests/

README.md

.env.example

=====

26. PYTHON LIBRARIES

=====

Use appropriate libraries such as:

fastapi
uvicorn
pydantic
sqlalchemy
python-multipart

pymupdf
python-docx
pytesseract
opencv-python
Pillow

pandas
numpy
scikit-learn
xgboost
joblib

shap

matplotlib
seaborn

reportlab

python-dateutil

Use only necessary dependencies.

=====

27. AI/NLP FEATURES

=====

Use NLP techniques for textual analysis.

Possible techniques:

TF-IDF
N-grams
tokenization
stopword handling
lemmatization where useful
text similarity

duplicate sentence detection

Use spaCy or NLTK where appropriate.

Do not over-engineer.

The project should remain understandable to college students presenting it in a viva.

=====

28. DATASET STRATEGY

=====

Design the application so that the training dataset can contain:

document text
document type
label
metadata features
structural features
consistency features

Example:

document_id,text,document_type,label

123,"Certificate issued to...",certificate,0

124,"Certificate issued to...",certificate,1

For demonstration, provide a small sample dataset ONLY if clearly marked:

"Demo dataset – replace with real/collected dataset before final evaluation."

Never present synthetic data as real-world benchmark data.

=====

29. IMPORTANT ACADEMIC FEATURE: COMPARATIVE ANALYSIS

=====

The report must contain a section:

"Comparative Performance Analysis of Machine Learning Models"

Compare:

Logistic Regression
Random Forest

XGBoost

Use the same train/test split and evaluation methodology.

Display:

Accuracy

Precision

Recall

F1

ROC-AUC

Also explain:

Why the model was selected

Advantages

Limitations

Computational cost

Interpretability

The application should automatically generate the comparison table from actual evaluation results.

=====
30. VISUALIZATIONS
=====

Generate visualizations dynamically.

Required:

1. Model Accuracy Comparison
2. Precision/Recall/F1 Comparison
3. Confusion Matrix for each model
4. ROC Curve
5. Risk Score Distribution
6. Document Classification Distribution
7. Marker Category Frequency
8. Top Feature Importance

Charts should use the same black/green design language.

=====
31. RESULT LOGIC
=====

The application should never simply say:

"100% Fake"

Instead:

LIKELY FAKE

Risk Score:

84/100

Confidence:

88%

Evidence:

- 2 date inconsistencies
- 1 numerical inconsistency
- 3 formatting anomalies
- unusual metadata
- ML models show elevated fraud probability

Then:

"Why this result?"

Explain the reasoning in simple language.

=====

32. MULTIPLE DOCUMENT COMPARISON

=====

Add an advanced feature:

"Compare Documents"

Allow user to upload two documents.

Compare:

- textual similarity
- common names
- dates
- numbers
- IDs
- document structure
- metadata
- formatting
- suspicious differences

Example:

Document A:
Certificate dated 12/03/2026

Document B:
Certificate dated 15/03/2026

Show:

"Potential inconsistency between documents."

This feature should be available from the dashboard.

=====

33. BATCH SCANNING

=====

Allow multiple documents to be uploaded.

Display:

File
Risk
Decision
Markers
Processing status

Allow sorting by:

Risk
Filename
Date
Document type

=====

34. SEARCHABLE MARKER DATABASE

=====

Create a marker rules configuration.

Example:

markers.json

Categories:

dates

numbers
text
formatting
metadata
structure
visual

This makes the project extensible.

Do not hard-code every rule into one enormous Python file.

=====

35. ERROR STATES

=====

Handle:

Unsupported file
Corrupted document
OCR unavailable
No text extracted
Password-protected PDF
Large file
Invalid image
Model not trained
Database error

Show user-friendly messages.

Example:

"No readable text was detected. The document may require OCR or may contain protected content."

=====

36. PERFORMANCE

=====

Normal document analysis should be asynchronous where necessary.

Show processing status.

Do not block the entire frontend.

For the mini-project, FastAPI BackgroundTasks can be used.

For heavier deployment, architecture should allow Celery/Redis later.

37. TESTING

Create tests for:

PDF extraction
DOCX extraction
TXT extraction
OCR preprocessing
date detection
number consistency
marker detection
feature extraction
model prediction
risk scoring
API endpoints

Use pytest.

38. README

Create an excellent README.

Include:

Project Overview
Problem Statement
Objectives
Features
System Architecture
Technology Stack
ML Models
Dataset
Feature Engineering
Evaluation Metrics
Installation
Running Backend
Running Frontend
Training Models
Using the Application
Project Structure
Screenshots placeholders
Future Scope
Limitations
Team Contribution

References

=====

39. VIVA-FRIENDLY ARCHITECTURE

=====

Make sure the entire architecture can be explained in a college viva.

Explain:

Input



Document Processing



OCR



Feature Extraction



Marker Detection



ML Prediction



Ensemble Risk Scoring



Explainability



Visualization

The developers should be able to explain every component.

Avoid adding unnecessary technologies simply to make the project look complicated.

=====

40. DESIGN PHILOSOPHY

=====

The final application should feel like:

"Document Forensics + AI Security Dashboard"

NOT:

"Generic AI chatbot"

Do not include a chatbot unless it has a real purpose.

No glassmorphism.

No excessive animations.

No giant glowing neon text.

No random AI-generated marketing language.

Use realistic labels and technical terminology.

Color palette:

#000000

#0A0A0A

#111111

#1A1A1A

green accent

white

gray

The interface should look consistent across:

Dashboard

Upload

Analysis

Model Performance

History

Reports

Comparison

=====

41. IMPORTANT REAL-WORLD LIMITATIONS

=====

The application must clearly communicate:

AI cannot guarantee whether a document is legally authentic.

A document may be genuine but contain unusual formatting.

A forged document may be sophisticated enough to bypass automated checks.

External verification through issuing authorities would be required for definitive authentication.

Therefore:

Use terms:

"Likely Genuine"

"Suspicious"
"Likely Fake"

instead of:

"100% Real"
"100% Fake"

=====

42. FINAL DELIVERABLE

=====

Generate:

1. Complete backend
2. Complete frontend
3. ML training pipeline
4. ML prediction pipeline
5. Marker detection engine
6. OCR pipeline
7. Database
8. Model evaluation
9. Visualization dashboard
10. Document comparison
11. Batch scanning
12. PDF report generation
13. Tests
14. README
15. .env.example
16. requirements.txt
17. Sample/demo dataset clearly labelled as demo data
18. Model performance dashboard
19. Explainable AI functionality

The application must run locally with:

Backend:
uvicorn app.main:app --reload

Frontend:
npm run dev

Provide clear setup instructions.

=====

43. DEVELOPMENT ORDER

=====

Implement in this order:

PHASE 1

Project structure + backend + frontend

PHASE 2

Document upload and extraction

PHASE 3

OCR

PHASE 4

Feature extraction

PHASE 5

Marker detection

PHASE 6

Dataset preprocessing

PHASE 7

Three ML models

PHASE 8

Evaluation and comparison

PHASE 9

Risk scoring + explainability

PHASE 10

Dashboard

PHASE 11

History/database

PHASE 12

PDF report

PHASE 13

Document comparison

PHASE 14

Batch scanning

PHASE 15

Testing + polishing

After each phase, ensure the application still runs.

=====

44. CODE QUALITY

=====

Write clean, modular Python.

Use:

type hints
docstrings
logging
error handling
Pydantic validation
environment variables
modular services

Avoid:

one giant Python file
hard-coded paths
hard-coded model results
fake metrics
fake AI responses
unnecessary dependencies

=====

45. FINAL QUALITY CHECK

=====

Before considering the project complete, verify:

- ☐ PDF upload works
- ☐ DOCX upload works
- ☐ TXT upload works
- ☐ Image upload works
- ☐ OCR works
- ☐ Complete document text is extracted
- ☐ Document statistics are calculated
- ☐ Markers are detected
- ☐ Date inconsistencies are detected
- ☐ Number inconsistencies are detected
- ☐ Structural anomalies are detected
- ☐ Metadata is extracted
- ☐ Feature vector is generated
- ☐ Logistic Regression works
- ☐ Random Forest works
- ☐ XGBoost works

- ☐ Models are evaluated
- ☐ Metrics are calculated
- ☐ Confusion matrices are generated
- ☐ ROC curves are generated
- ☐ Predictions are explainable
- ☐ Risk score is generated
- ☐ Dashboard works
- ☐ History works
- ☐ PDF report works
- ☐ Batch scanning works
- ☐ Document comparison works
- ☐ Error handling works
- ☐ README is complete

Do not stop at creating a UI mockup.

Build the actual functional application.

The final result should be suitable for demonstration as a college AIML mini-project and should clearly demonstrate:

DOCUMENT PROCESSING

+

OCR

+

NLP

+

FEATURE ENGINEERING

+

MACHINE LEARNING

+

EXPLAINABLE AI

+

VISUALIZATION

+

DECISION SUPPORT